

Python collections.Counter 📌 What is Counter?

Counter is a class in the collections module in Python. It is a **dictionary subclass** used for **counting hashable objects**. It helps you quickly tally elements like items in a list, characters in a string, or values in a tuple.

```
from collections import Counter
```

Syntax

```
Counter([iterable-or-mapping])
```

You can pass an **iterable** (like a list, string, or tuple), a **dictionary**, or use **keyword arguments**.

Input Types and Examples

1. List

```
Counter(['a', 'b', 'a', 'c'])
```

```
# Output: Counter({'a': 2, 'b': 1, 'c': 1})
```

2. String

```
Counter("banana")
```

```
# Output: Counter({'a': 3, 'n': 2, 'b': 1})
```

3. Tuple

```
Counter(('dog', 'cat', 'dog'))
```

```
# Output: Counter({'dog': 2, 'cat': 1})
```

4. Dictionary

```
Counter({'x': 4, 'y': 2})
```

```
# Output: Counter({'x': 4, 'y': 2})
```

5. Keyword Arguments

```
Counter(a=3, b=2)
```

```
# Output: Counter({'a': 3, 'b': 2})
```

Main Methods of Counter

► **most_common([n])**

Returns the most common elements in descending order.

```
c = Counter("abracadabra")
```

```
c.most_common(2)
```

```
# Output: [('a', 5), ('b', 2)]
```

► **elements()**

Returns an iterator with elements repeated by their count.

```
c = Counter(a=2, b=1)
```

```
list(c.elements())
```

```
# Output: ['a', 'a', 'b']
```

► **update()**

Adds counts from another iterable or mapping.

```
c = Counter(a=1, b=2)
```

```
c.update(['a', 'b', 'c'])
```

```
# Output: Counter({'b': 3, 'a': 2, 'c': 1})
```

► **subtract()**

Subtracts counts, allowing negative values.

```
c = Counter(a=4, b=2)
```

```
c.subtract({'a': 1, 'b': 3})
```

```
# Output: Counter({'a': 3, 'b': -1})
```

+ Counter Arithmetic Operations

Add Counters

```
Counter(a=2) + Counter(a=1, b=2)
```

```
# Output: Counter({'a': 3, 'b': 2})
```

Subtract Counters (no negatives)

```
Counter(a=3, b=1) - Counter(a=1, b=2)
```

```
# Output: Counter({'a': 2})
```

Intersection (&)

Keeps the **minimum** of counts.

```
Counter(a=3, b=1) & Counter(a=1, b=2)
```

```
# Output: Counter({'a': 1, 'b': 1})
```

Union (|)

Keeps the **maximum** of counts.

```
Counter(a=3, b=1) | Counter(a=1, b=2)
```

```
# Output: Counter({'a': 3, 'b': 2})
```

Real-World Example: Word Frequency Counter

```
text = "this is a test this is only a test"
```

```
words = text.split()
```

```
counter = Counter(words)
```

```
print(counter.most_common())
```

```
# Output: [('this', 2), ('is', 2), ('a', 2), ('test', 2), ('only', 1)]
```

Note

- Counts can be **0 or negative**, especially after subtraction.
- Counter() returns 0 for missing keys instead of KeyError.
- It is optimized for performance when counting elements.